Introduction:

The purpose of this project was to implement database with visual programming to make a

movie browser app which:

- Has a certain number of movies stored in the database
- Allows the user to access those lists
- Allows the user to add movies to their watch-later list

The project was developed in an Arch Linux system. Unfortunately, the official Arch repository that the Arch package manager ‘pacman’ uses does not have Oracle’s MySQL, therefore the project has a MariaDB (a community fork of MySQL compatible with JDBC) relational database connecting with JDBC (java database connection).

Subprojects Present:

To simulate a simple server side and client side connection, the project contains two subprojects within it:

1. DatabaseServerProject
2. ClientProject

The former is intializes the database, adds movies to it while the latter can only read from the main database table, namely ‘movies’ to load the movies available for viewing and adds/removes movies to/from a ‘watch_later’ folder.

Structure of the file system:

1. DatabaseServerProject

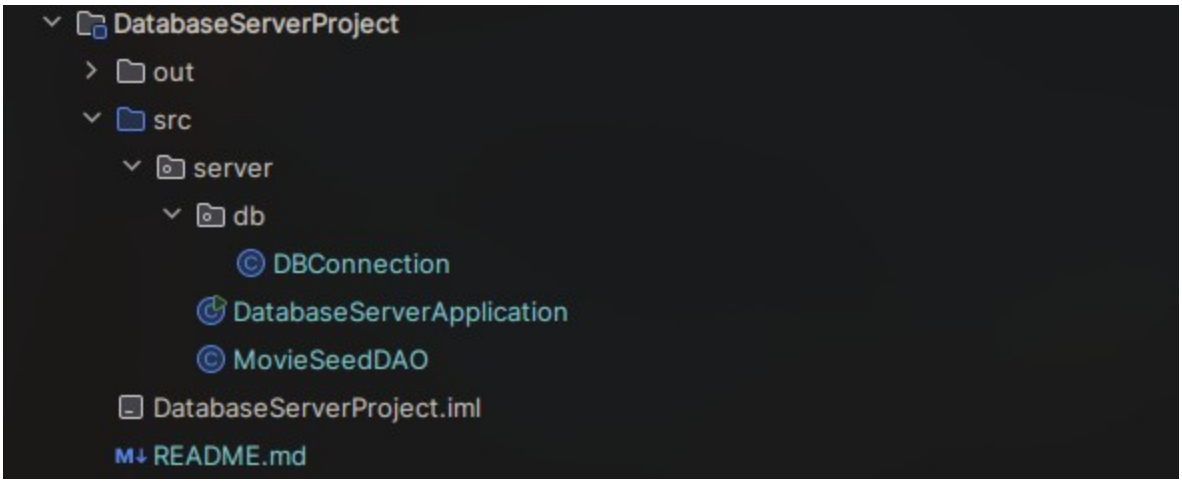


Fig: file structure of DatabaseServerProject

DBConnection.java is responsible for creating the connection between the Java project and the MariaDB database. It stores database settings like host, port, database name, username, and password, builds the JDBC URL, for example jdbc:mysql://127.0.0.1:3306/vpl_lab, opens a database connection using DriverManager.getConnection() and lets DAO classes reuse the same connection logic instead of repeating it everywhere.

DatabaseServerApplication.java is the launcher for the database setup project. When you run it, it creates a MovieSeedDAO, starts the database initialization process, counts how many movies are available, and prints whether the database is ready. If something fails, it prints the database connection error.

MovieSeedDAO.java does the actual database work. It creates the vpl_lab database if needed, creates the movies and watch_later tables, inserts the initial list of movie records, and provides countMovies() to verify how many movies exist. It uses INSERT IGNORE so running the setup multiple times does not duplicate movies.

2. ClientProject

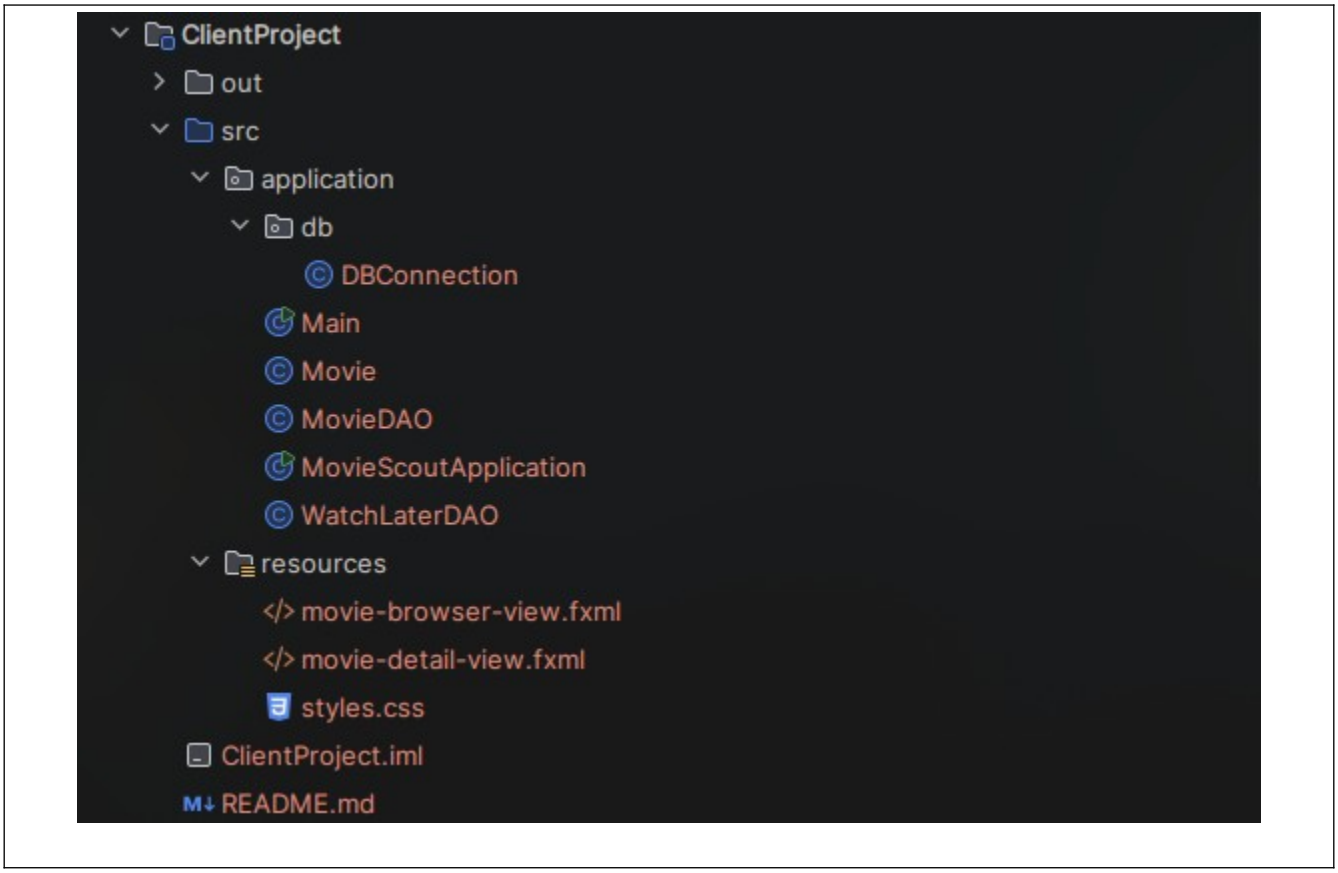


Fig: file structure of ClientProject

In the client project, **DBConnection.java** connects the JavaFX app to the MariaDB database. Main.java starts the JavaFX application. It simply launches MovieScoutApplication.

MovieScoutApplication.java is the main UI client class. It handles functions such as loading the FXML files to from the GUI and calls DAO classes to load/update the data in the database.

Movie.java is a model class for one movie. It stores fields like id, title, genres, cast, durationMinutes, imdbRating, summary, posterUrl, and watchLater. This class replicates the database table 'movies'.

MovieDAO.java handles movie-related database queries. It counts movies, loads movies, searches by title, filters by genre, filters Watch Later movies, loads all genres, and finds one movie by ID.

WatchLaterDAO.java handles the Watch Later database actions. It adds a movie to watch_later and removes a movie from it.

Additionally, the resources section contains the FXML files required to load the UI.

Schema

```
MariaDB [vpl_lab]> SHOW TABLES;
+--------------------+
| Tables_in_vpl_lab  |
+--------------------+
| movies             |
| watch_later        |
+--------------------+
2 rows in set (0.000 sec)
```

Fig: Schema

Table Design

1. ‘movies’ table

```
MariaDB [vpl_lab]> describe movies;
+------------------+--------------+------+-----+---------+----------------+
| Field            | Type         | Null | Key | Default | Extra          |
+------------------+--------------+------+-----+---------+----------------+
| id               | int(11)      | NO   | PRI | NULL    | auto_increment |
| title            | varchar(160) | NO   | UNI | NULL    |                |
| genres           | varchar(180) | NO   |     | NULL    |                |
| cast_members     | varchar(260) | NO   |     | NULL    |                |
| duration_minutes | int(11)      | NO   |     | NULL    |                |
| imdb_rating      | decimal(3,1) | NO   |     | NULL    |                |
| summary          | text         | NO   |     | NULL    |                |
| poster_url       | varchar(500) | NO   |     | NULL    |                |
+------------------+--------------+------+-----+---------+----------------+
8 rows in set (0.005 sec)
```

Fig: The columns of the table with the data type of each columns.

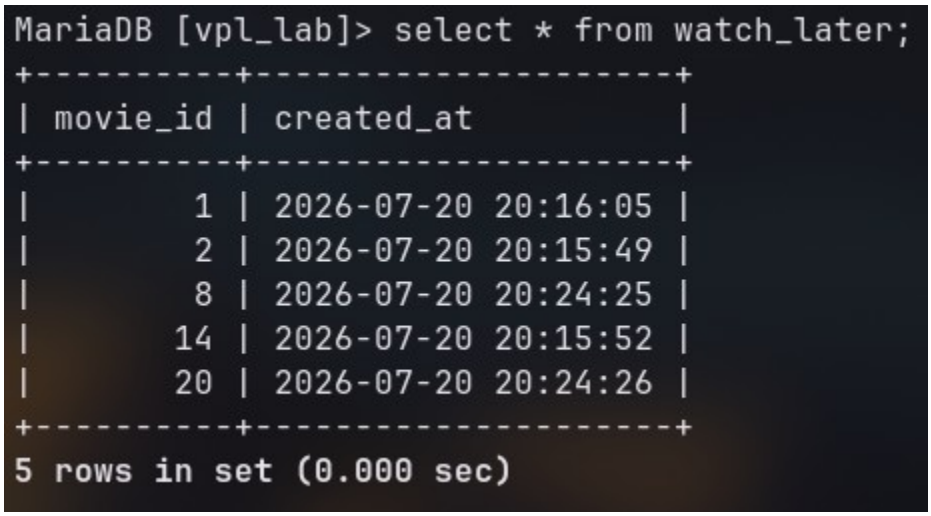
```
MariaDB [vpl_lab]> SELECT id, title, genres FROM movies;
+----+---------------------------------------------+------------------------------+
| id | title                                       | genres                       |
+----+---------------------------------------------+------------------------------+
|  1 | How to Train Your Dragon                    | Animation, Adventure, Family |
|  2 | Demon Slayer: Kimetsu no Yaiba Infinity Castle | Animation, Action, Fantasy |
|  3 | Lilo & Stitch                               | Adventure, Comedy, Family    |
|  4 | M3GAN 2.0                                   | Horror, Sci-Fi, Thriller     |
|  5 | Inception                                   | Action, Sci-Fi, Thriller     |
|  6 | Interstellar                                | Adventure, Drama, Sci-Fi     |
|  7 | The Dark Knight                             | Action, Crime, Drama         |
|  8 | Spider-Man: Into the Spider-Verse           | Animation, Action, Adventure |
|  9 | Dune: Part Two                              | Adventure, Drama, Sci-Fi     |
| 10 | Inside Out 2                                | Animation, Comedy, Family    |
| 11 | The Batman                                  | Action, Crime, Mystery       |
| 12 | Avatar: The Way of Water                    | Action, Adventure, Fantasy   |
| 13 | Toy Story                                   | Animation, Adventure, Comedy |
| 14 | Spirited Away                               | Animation, Adventure, Fantasy |
| 15 | Parasite                                    | Drama, Thriller              |
| 16 | Oppenheimer                                 | Biography, Drama, History    |
| 17 | Barbie                                      | Adventure, Comedy, Fantasy   |
| 18 | John Wick: Chapter 4                        | Action, Crime, Thriller      |
| 19 | Godzilla x Kong: The New Empire             | Action, Adventure, Sci-Fi    |
| 20 | Mission: Impossible - Dead Reckoning        | Action, Adventure, Thriller  |
+----+---------------------------------------------+------------------------------+
20 rows in set (0.000 sec)
```

Fig: Query output showing the selected columns.

2. ‘watch_later’ table

```
MariaDB [vpl_lab]> describe watch_later;
+------------+-----------+------+-----+---------------------+-------+
| Field      | Type      | Null | Key | Default             | Extra |
+------------+-----------+------+-----+---------------------+-------+
| movie_id   | int(11)   | NO   | PRI | NULL                |       |
| created_at | timestamp | YES  |     | current_timestamp() |       |
+------------+-----------+------+-----+---------------------+-------+
2 rows in set (0.001 sec)
```

Fig: The columns of the table with the data type of each columns.



```
MariaDB [vpl_lab]> select * from watch_later;
+----------+---------------------+
| movie_id | created_at          |
+----------+---------------------+
|        1 | 2026-07-20 20:16:05 |
|        2 | 2026-07-20 20:15:49 |
|        8 | 2026-07-20 20:24:25 |
|       14 | 2026-07-20 20:15:52 |
|       20 | 2026-07-20 20:24:26 |
+----------+---------------------+
5 rows in set (0.000 sec)
```

Fig: Query output showing all columns.

DBConnection.java

```
public class DBConnection {
    private static final String DATABASE = "vpl_lab";
    private static final String SERVER_URL = "jdbc:mysql://127.0.0.1:3306"
            + "?connectTimeout=5000&socketTimeout=5000";
    private static final String DATABASE_URL = "jdbc:mysql://127.0.0.1:3306/" +
DATABASE
            + "?connectTimeout=5000&socketTimeout=5000";
    private static final String USER = "root";
    private static final String PASSWORD = "mysql";

    public static Connection getServerConnection() throws SQLException {
        return DriverManager.getConnection(SERVER_URL, USER, PASSWORD);
    }

    public static Connection getConnection() throws SQLException {
        return DriverManager.getConnection(DATABASE_URL, USER, PASSWORD);
    }

    public static String getSettingsDescription() {
        return USER + "@127.0.0.1:3306/" + DATABASE;
    }

    public static String getDatabaseName() {
        return DATABASE;
    }
}
```

Contains the database configurations like the name “vpl_lab”, the url “jdbc:mysql://127.0.0.1” (since the server is running locally) and the default port number that the process is using ‘3306’.

Movie.java

```
public class Movie {
    private final int id;
    private final String title;
    private final String genres;
    private final String cast;
    private final int durationMinutes;
    private final double imdbRating;
    private final String summary;
    private final String posterUrl;
    private final boolean watchLater;

    public Movie(
            int id,
            String title,
            String genres,
            String cast,
            int durationMinutes,
            double imdbRating,
            String summary,
            String posterUrl,
            boolean watchLater
    ) {
        this.id = id;
        this.title = title;
        this.genres = genres;
        this.cast = cast;
        this.durationMinutes = durationMinutes;
        this.imdbRating = imdbRating;
        this.summary = summary;
        this.posterUrl = posterUrl;
        this.watchLater = watchLater;
    }
```

Model class containing the attributes of the ‘movies’ table.

MovieScoutApplication.java

```
public class MovieScoutApplication extends Application {
 private final MovieDAO movieDAO = new MovieDAO();
 private final WatchLaterDAO watchLaterDAO = new WatchLaterDAO();

 private BorderPane root;
 private FlowPane movieGrid;
 private TextField searchField;
 private ComboBox<String> genreBox;
 private Button allButton;
 private Button watchButton;
 private Button themeButton;
 private Label statusLabel;
 private boolean watchLaterOnly;
 private boolean darkMode = true;

 @Override
 public void start(Stage stage) {...}

    private BorderPane loadMainView() {...}

    private void loadGenres() {...}

    private void refreshMovies() {
        if (movieGrid == null || searchField == null || genreBox == null) {
            return;
        }

        try {
            List<Movie> movies = movieDAO.findMovies(
                    searchField.getText(),
                    genreBox.getValue(),
                    watchLaterOnly
            );

movieGrid.getChildren().setAll(movies.stream().map(this::createMovieCard).toList()
);
            statusLabel.setText(movies.size() + " movie" + (movies.size() == 1 ?
""     : "s") + " shown");
        } catch (SQLException exception) {
            showError("Could not load movies", exception);
        }
    }


    private VBox createMovieCard(Movie movie) {
        StackPane posterPane = createPoster(movie, 214, 300);
```

```
        Label title = new Label(movie.getTitle());
        title.getStyleClass().add("movie-title");
        title.setWrapText(true);

        Label year = new Label("2025");
        year.getStyleClass().add("movie-year");

        FlowPane chips = new FlowPane();
        chips.getStyleClass().add("genre-chip-row");
        chips.setHgap(6);
        chips.setVgap(6);
        for (String genre : movie.getGenres().split(",")) {
            Label chip = new Label(genre.trim());
            chip.getStyleClass().add("genre-chip");
            chips.getChildren().add(chip);
        }

        Label star = new Label("★");
        star.getStyleClass().add("rating-star");

        Label rating = new Label(String.format("%.1f", movie.getImdbRating()));
        rating.getStyleClass().add("rating-label");

        Label votes = new Label(voteCount(movie) + " votes");
        votes.getStyleClass().add("vote-label");

        HBox ratingRow = new HBox(5, star, rating, votes);
        ratingRow.getStyleClass().add("rating-row");
        ratingRow.setAlignment(Pos.CENTER_LEFT);

        VBox body = new VBox(6, title, year, chips);
        body.getStyleClass().add("movie-card-body");
        VBox.setVgrow(body, javafx.scene.layout.Priority.ALWAYS);

        VBox card = new VBox(10, posterPane, body, ratingRow);
        card.getStyleClass().add("movie-card");
        card.setOnMouseClicked(event -> showDetails(movie.getId()));
        return card;
    }


    private int voteCount(Movie movie) {}

    private StackPane createPoster(Movie movie, double width, double height) {
        ImageView poster = new ImageView();
        poster.setFitWidth(width);
        poster.setFitHeight(height);
        poster.setPreserveRatio(false);
```

```
        poster.setImage(new Image(movie.getPosterUrl(), width, height, false,
true, true));

        Label fallback = new Label(movie.getTitle());
        fallback.getStyleClass().add("poster-fallback");
        fallback.setWrapText(true);
        fallback.setAlignment(Pos.CENTER);
        fallback.setMaxWidth(width - 24);

        StackPane posterPane = new StackPane(fallback, poster);
        posterPane.getStyleClass().add("poster-pane");
        posterPane.setPrefSize(width, height);
        posterPane.setMinSize(width, height);
        posterPane.setMaxSize(width, height);
        return posterPane;
    }


    private void showDetails(int movieId)  {
        try {
            Movie movie = movieDAO.findById(movieId);

            Stage dialog = new Stage();
            dialog.initModality(Modality.APPLICATION_MODAL);
            dialog.setTitle(movie.getTitle());

            FXMLLoader loader = new
FXMLLoader(MovieScoutApplication.class.getResource("/resources/movie-detail-
view.fxml"));
            HBox layout = loader.load();
            layout.getStyleClass().add(darkMode ? "dark" : "light");
            Map<String, Object> controls = loader.getNamespace();

            StackPane posterSlot = getControl(controls, "posterSlot",
StackPane.class);
            Label title = getControl(controls, "detailTitle", Label.class);
            Label meta = getControl(controls, "detailMeta", Label.class);
            Label genres = getControl(controls, "detailGenres", Label.class);
            Label cast = getControl(controls, "detailCast", Label.class);
             Label summary = getControl(controls, "detailSummary", Label.class);
            Button watchLaterButton = getControl(controls,
"detailWatchLaterButton", Button.class);
            Button closeButton = getControl(controls, "detailCloseButton",
Button.class);

            posterSlot.getChildren().setAll(createPoster(movie, 220, 320));
            title.setText(movie.getTitle());
            title.setWrapText(true);
```

```
            meta.setText(movie.getDurationMinutes() + " minutes  |  IMDb " +
String.format("%.1f", movie.getImdbRating()));
            genres.setText(movie.getGenres());
            genres.setWrapText(true);

            cast.setText("Cast: " + movie.getCast());
            cast.setWrapText(true);

            summary.setText(movie.getSummary());
            summary.setWrapText(true);

             watchLaterButton.setText(movie.isWatchLater() ? "Remove from Watch
Later" : "Add to Watch Later");
            watchLaterButton.getStyleClass().add(movie.isWatchLater() ? "danger-
button" : "primary-button");
            watchLaterButton.setOnAction(event -> {
                toggleWatchLater(movie);
                dialog.close();
            });

            closeButton.setOnAction(event -> dialog.close());

            Scene scene = new Scene(layout, 740, 420);
            dialog.setScene(scene);
            dialog.showAndWait();
        } catch (IOException | SQLException exception) {
            showError("Could not load movie details", exception);
        }
    }


    private void toggleWatchLater(Movie movie) {...}

    private void toggleTheme() {...}

    private void showFatalScreen(Stage stage, SQLException exception) {...}

    private void showError(String title, Exception exception){...}

};
```

This is the main UI file that loads the dynamically loads the UI components using the FXMLfiles and sets/changes scenes files and displays them to the user.

refreshMovies() loads movie data from the database and replaces the contents of movieGrid with newly created movie cards.

createMovieCard(Movie movie) dynamically creates each movie card using JavaFX classes like VBox, Label, FlowPane, HBox, and StackPane.

It creates:

- poster image
- movie title
- year label
- genre chips
- rating row
- clickable card

createPoster(Movie movie, double width, double height) Dynamically creates the poster section using ImageView, Label, and StackPane.

showDetails(int movieId) loads the detail FXML, then dynamically fills its fields with the selected movie's data.

MovieSeedDAO.java

```
public class MovieSeedDAO {
    public void initializeDatabase() throws SQLException {
        createDatabase();
        createTables();
        seedMovies();
    }

    public int countMovies() throws SQLException {
        String sql = "SELECT COUNT(*) AS movie_count FROM movies";

        try (Connection connection = DBConnection.getConnection();
             PreparedStatement statement = connection.prepareStatement(sql);
             ResultSet resultSet = statement.executeQuery()) {
            if (resultSet.next()) {
                return resultSet.getInt("movie_count");
            }
            return 0;
        }
    }

    private void createDatabase() throws SQLException {
        String sql = "CREATE DATABASE IF NOT EXISTS `" +
DBConnection.getDatabaseName() + "` "
                + "CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci";

        try (Connection connection = DBConnection.getServerConnection();
             Statement statement = connection.createStatement()) {
            statement.execute(sql);
        }
    }

    private void createTables() throws SQLException {
        String moviesSql = """
                CREATE TABLE IF NOT EXISTS movies (
                    id INT AUTO_INCREMENT PRIMARY KEY,
                    title VARCHAR(160) NOT NULL UNIQUE,
                    genres VARCHAR(180) NOT NULL,
                    cast_members VARCHAR(260) NOT NULL,
                    duration_minutes INT NOT NULL,
                    imdb_rating DECIMAL(3,1) NOT NULL,
                    summary TEXT NOT NULL,
                    poster_url VARCHAR(500) NOT NULL
                )
                """;
```

```
        String watchLaterSql = """
                CREATE TABLE IF NOT EXISTS watch_later (
                    movie_id INT PRIMARY KEY,
                    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                    CONSTRAINT fk_watch_later_movie
                        FOREIGN KEY (movie_id) REFERENCES movies(id)
                        ON DELETE CASCADE
                )
                """;

        try (Connection connection = DBConnection.getConnection();
             Statement statement = connection.createStatement()) {
            statement.execute(moviesSql);
            statement.execute(watchLaterSql);
        }
    }

    private void seedMovies() throws SQLException {
        String sql = """
                INSERT IGNORE INTO movies
                (title, genres, cast_members, duration_minutes, imdb_rating,
summary, poster_url)
                VALUES (?, ?, ?, ?, ?, ?, ?)
                """;

        try (Connection connection = DBConnection.getConnection();
             PreparedStatement statement = connection.prepareStatement(sql)) {
            for (Object[] movie : seedData()) {
                statement.setString(1, (String) movie[0]);
                statement.setString(2, (String) movie[1]);
                statement.setString(3, (String) movie[2]);
                statement.setInt(4, (Integer) movie[3]);
                statement.setDouble(5, (Double) movie[4]);
                statement.setString(6, (String) movie[5]);
                statement.setString(7, (String) movie[6]);
                statement.addBatch();
            }
            statement.executeBatch();
        }
    }

    private List<Object[]> seedData() {...}

    private Object[] movie(String title, String genres, String cast, int duration,
double rating, String summary, String posterUrl) {}
}
```

This is the server side file that seeds the movies and loads them into the database for viewers to browse.

countMovies() return the number of entries in the ‘movies’ table. It is mainly used for verification purposes: to test the connection and existence of the said table before any further queries are made.

createDatabase() creates the mariaDB database with the given configurations.

createTable() carries out the SQL commands required to form the schema.

seedMovies() inputs movies into the ‘movies’ table from the movies array.

MovieDAO.java

```
public class MovieDAO {
    public int countMovies() throws SQLException {
        String sql = "SELECT COUNT(*) AS movie_count FROM movies";

        try (Connection connection = DBConnection.getConnection();
             PreparedStatement statement = connection.prepareStatement(sql);
             ResultSet resultSet = statement.executeQuery()) {
            if (resultSet.next()) {
                return resultSet.getInt("movie_count");
            }
            return 0;
        }
    }

    public List<Movie> findMovies(String searchText, String genre, boolean
watchLaterOnly) throws SQLException {
        StringBuilder sql = new StringBuilder("""
                SELECT m.id, m.title, m.genres, m.cast_members,
m.duration_minutes,
                       m.imdb_rating, m.summary, m.poster_url,
                       CASE WHEN wl.movie_id IS NULL THEN FALSE ELSE TRUE END AS
watch_later
                FROM movies m
                LEFT JOIN watch_later wl ON wl.movie_id = m.id
                WHERE LOWER(m.title) LIKE LOWER(?)
                """);

        List<Object> values = new ArrayList<>();
        values.add("%" + searchText.trim() + "%");

        if (genre != null && !genre.equals("All Genres")) {
            sql.append(" AND LOWER(m.genres) LIKE LOWER(?)");
            values.add("%" + genre + "%");
        }
```

```
        if (watchLaterOnly) {
            sql.append(" AND wl.movie_id IS NOT NULL");
        }

        sql.append(" ORDER BY m.imdb_rating DESC, m.title");

        try (Connection connection = DBConnection.getConnection();
             PreparedStatement statement =
connection.prepareStatement(sql.toString())) {
            bindValues(statement, values);
            try (ResultSet resultSet = statement.executeQuery()) {
                List<Movie> movies = new ArrayList<>();
                while (resultSet.next()) {
                    movies.add(toMovie(resultSet));
                }
                return movies;
            }
        }
    }

    public List<String> findGenres() throws SQLException {
        String sql = "SELECT genres FROM movies ORDER BY genres";

        try (Connection connection = DBConnection.getConnection();
             PreparedStatement statement = connection.prepareStatement(sql);
             ResultSet resultSet = statement.executeQuery()) {
            List<String> genres = new ArrayList<>();
            while (resultSet.next()) {
                for (String genre : resultSet.getString("genres").split(",")) {
                    String cleanGenre = genre.trim();
                    if (!cleanGenre.isBlank() && !genres.contains(cleanGenre)) {
                        genres.add(cleanGenre);
                    }
                }
            }
            genres.sort(String::compareToIgnoreCase);
            return genres;
        }
    }
```

```
      public Movie findById(int movieId) throws SQLException {
        String sql = """
                SELECT m.id, m.title, m.genres, m.cast_members,
m.duration_minutes,
                       m.imdb_rating, m.summary, m.poster_url,
                       CASE WHEN wl.movie_id IS NULL THEN FALSE ELSE TRUE END AS
watch_later
                FROM movies m
                LEFT JOIN watch_later wl ON wl.movie_id = m.id
                WHERE m.id = ?
                """;

        try (Connection connection = DBConnection.getConnection();
             PreparedStatement statement = connection.prepareStatement(sql)) {
            statement.setInt(1, movieId);
            try (ResultSet resultSet = statement.executeQuery()) {
                if (resultSet.next()) {
                    return toMovie(resultSet);
                }
                throw new SQLException("Movie not found.");
            }
        }
    }

    private void bindValues(PreparedStatement statement, List<Object> values)
throws SQLException {
        for (int index = 0; index < values.size(); index++) {
            statement.setObject(index + 1, values.get(index));
        }
    }

    private Movie toMovie(ResultSet resultSet) throws SQLException {
        return new Movie(
                resultSet.getInt("id"),
                resultSet.getString("title"),
                resultSet.getString("genres"),
                resultSet.getString("cast_members"),
                resultSet.getInt("duration_minutes"),
                resultSet.getDouble("imdb_rating"),
                resultSet.getString("summary"),
                resultSet.getString("poster_url"),
                resultSet.getBoolean("watch_later")
        );
    }
}
```

countMovies() returns the number of entries in the movies table. It is mainly used to verify that the database connection works and that the movies table exists.

findMovies() retrieves movies from the database based on the current search text, selected genre, and Watch Later filter. It uses a LEFT JOIN with the watch_later table to also check whether each movie is saved in Watch Later.

findGenres() retrieves all genre strings from the movies table, separates comma-listed genres, removes duplicates, sorts them, and returns them for the genre dropdown.

findById() retrieves one specific movie using its id. It is used when a movie card is clicked to show the movie details page/dialog.

bindValues() attaches the search/filter values to the prepared SQL statement. This keeps the query safer and avoids manually inserting user input into SQL.

toMovie() converts one database result row into a Movie object. It reads columns like title, genres, cast, rating, summary, poster URL, and Watch Later status.

WatchLaterDAO.java

```
public class WatchLaterDAO {
    public void addMovie(int movieId) throws SQLException {
        String sql = "INSERT IGNORE INTO watch_later (movie_id) VALUES (?)";

        try (Connection connection = DBConnection.getConnection();
             PreparedStatement statement = connection.prepareStatement(sql)) {
            statement.setInt(1, movieId);
            statement.executeUpdate();
        }
    }

    public void removeMovie(int movieId) throws SQLException {
        String sql = "DELETE FROM watch_later WHERE movie_id = ?";

        try (Connection connection = DBConnection.getConnection();
             PreparedStatement statement = connection.prepareStatement(sql)) {
            statement.setInt(1, movieId);
            statement.executeUpdate();
        }
    }
}
```

addMovie() adds a movie into the ‘watch_later’ table of the database.

removeMovie() removes a movie from the ‘watch_later’ table of the database.

FXML Files

Movie-browser-view.fxml

```
<BorderPane xmlns="http://javafx.com/javafx/21"
            xmlns:fx="http://javafx.com/fxml/1"
            prefHeight="720.0"
            prefWidth="1040.0"
            stylesheets="@styles.css">
    <styleClass>
        <String fx:value="app-root" />
        <String fx:value="dark" />
    </styleClass>

    <top>
        <HBox alignment="CENTER_LEFT" spacing="12.0">
            <styleClass>
                <String fx:value="top-bar" />
            </styleClass>

            <Label alignment="CENTER" text="M">
                <styleClass>
                    <String fx:value="brand-icon" />
                </styleClass>
            </Label>

            <Label text="Movie Browser">
                <styleClass>
                    <String fx:value="brand-title" />
                </styleClass>
            </Label>

            <Button fx:id="themeButton" text="Dark">
                <styleClass>
                    <String fx:value="pill-button" />
                </styleClass>
            </Button>

            <Button fx:id="allButton" text="All">
                <styleClass>
                    <String fx:value="nav-button" />
                    <String fx:value="active" />
                </styleClass>
            </Button>

            <Button fx:id="watchButton" text="Watch Later">
                <styleClass>
```

```
                <String fx:value="nav-button" />
            </styleClass>
        </Button>

        <HBox HBox.hgrow="ALWAYS" />

        <TextField fx:id="searchField" promptText="Search movie...">
            <styleClass>
                <String fx:value="search-field" />
            </styleClass>
        </TextField>

        <ComboBox fx:id="genreBox" promptText="All Genres">
            <styleClass>
                <String fx:value="genre-box" />
            </styleClass>
        </ComboBox>
    </HBox>
</top>

<center>
    <VBox spacing="16.0">
        <styleClass>
            <String fx:value="content-panel" />
        </styleClass>
        <padding>
            <Insets bottom="22.0" left="34.0" right="22.0" top="28.0" />
        </padding>

        <Label text="Popular movies">
            <styleClass>
                <String fx:value="section-title" />
            </styleClass>
        </Label>

        <ScrollPane fitToWidth="true">
            <styleClass>
                <String fx:value="catalog-scroll" />
            </styleClass>
            <content>
                <FlowPane fx:id="movieGrid" hgap="18.0" vgap="22.0">
                    <styleClass>
                        <String fx:value="movie-grid" />
                    </styleClass>
                </FlowPane>
            </content>
        </ScrollPane>
    </VBox>
</center>
```

```
    <bottom>
        <HBox alignment="CENTER_LEFT">
            <styleClass>
                <String fx:value="status-bar" />
            </styleClass>
            <Label fx:id="statusLabel" text="Ready">
                <styleClass>
                    <String fx:value="status-label" />
                </styleClass>
            </Label>
        </HBox>
    </bottom>
</BorderPane>
```

Movie-detail-view.fxml

```
<HBox xmlns="http://javafx.com/javafx/21"
      xmlns:fx="http://javafx.com/fxml/1"
      alignment="TOP_LEFT"
      spacing="22.0"
      stylesheets="@styles.css">
    <styleClass>
        <String fx:value="detail-dialog" />
    </styleClass>
    <padding>
        <Insets bottom="24.0" left="24.0" right="24.0" top="24.0" />
    </padding>

    <StackPane fx:id="posterSlot" prefHeight="320.0" prefWidth="220.0" />

    <VBox alignment="TOP_LEFT" spacing="12.0" HBox.hgrow="ALWAYS">
        <Label fx:id="detailTitle" wrapText="true">
            <styleClass>
                <String fx:value="detail-title" />
            </styleClass>
        </Label>

        <Label fx:id="detailMeta">
            <styleClass>
                <String fx:value="detail-meta" />
            </styleClass>
        </Label>

        <Label fx:id="detailGenres" wrapText="true">
```

```
            <styleClass>
                <String fx:value="chip-line" />
            </styleClass>
        </Label>

        <Separator />

        <Label fx:id="detailCast" wrapText="true">
            <styleClass>
                <String fx:value="detail-body" />
            </styleClass>
        </Label>

        <Label fx:id="detailSummary" wrapText="true">
            <styleClass>
                <String fx:value="detail-body" />
            </styleClass>
        </Label>

        <HBox alignment="CENTER_LEFT" spacing="10.0">
            <Button fx:id="detailWatchLaterButton" text="Add to Watch Later" />
            <Button fx:id="detailCloseButton" text="Close">
                <styleClass>
                    <String fx:value="small-button" />
                </styleClass>
            </Button>
        </HBox>
    </VBox>
</HBox>
```